

Machine Learning

1. Gradient Descent for Multiple Variables

1.1 Definition

Gradient Descent is an optimisation algorithm used in Machine Learning to minimise the cost function (error). When a problem has multiple input features, Multivariable Gradient Descent is used to find the best values of all parameters simultaneously.

1.2 Why Multiple Variables Are Needed

Real-world outcomes depend on more than one factor. House price depends on size, location, and number of bedrooms — not just one variable. A single-variable model is too simplistic to capture such relationships.

1.3 Single Variable vs Multiple Variable Regression

Aspect	Single Variable	Multiple Variable
Input features	1 feature (x)	2 or more features ($x_1, x_2, \dots$)
Hypothesis	$h(x) = \theta_0 + \theta_1 x$	$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots$
Parameters	θ_0, θ_1	$\theta_0, \theta_1, \theta_2, \dots, \theta_n$
Complexity	Simple, one slope	More complex, multiple slopes
Use case	One input $\rightarrow$ one output	Multiple inputs $\rightarrow$ one output

1.4 Applications of Gradient Descent

- Linear Regression
- Logistic Regression
- Neural Networks
- Deep Learning

2. Linear Regression with Multiple Variables

2.1 Multiple Features

Multivariate Linear Regression uses more than one input feature to predict a single continuous output. Each feature contributes independently to the final prediction.

2.2 House Price Prediction Example

We predict house price using two input features:

- x_1 = House size (square feet)
- x_2 = Number of bedrooms
- y = House price (output / target)

2.3 The Multivariable Hypothesis Function

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

2.4 Term Definitions

Symbol	Name	Meaning
$h_{\theta}(x)$	Predicted value	The model output — what the model guesses
θ_0	Bias / Intercept	Constant term; shifts the prediction line up or down
θ_1, θ_2	Weights / Parameters	How strongly each feature influences the prediction
x_1	Feature 1	First input variable (e.g., house size)
x_2	Feature 2	Second input variable (e.g., number of bedrooms)

Step-by-Step Example

Suppose we want to predict exam score using:

Hours Studied (x_1)	Sleep Hours (x_2)	Score (y)
1	6	50
2	5	55
3	8	70

So: $m = 3$

Hypothesis Function $\gg h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$

Assume initial values:

- $\theta_0 = 0$
 - $\theta_1 = 0$
 - $\theta_2 = 0$
- and Learning rate:
- $\alpha = 0.01$

Figure 1 — Step-by-step example with two features (hours studied & sleep hours $\rightarrow$ exam score)

3. Hypothesis Function

3.1 General Form (n Features)

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

3.2 Components Explained

Features

Features are the input variables given to the model. Each feature represents one measurable property of a data point (e.g., house size, number of rooms, distance from city centre).

In vector notation, all features are collected as:

$$x_1, x_2, \dots, x_n$$

Weights / Parameters

Weights (also called parameters or coefficients) are numbers the model learns during training. A large weight means the corresponding feature has a strong influence on the prediction. All weights together are:

$$\theta_1, \theta_2, \dots, \theta_n$$

Bias Term θ_0

The bias term is an extra constant added to the equation. Without it, the hyperplane of predictions would always pass through the origin, which is almost never correct for real data. The bias shifts the prediction surface freely.

Memory rule: ϑ_0 is the y-intercept. $\vartheta_1, \vartheta_2, \dots$ are the slopes for each feature. Together they define the prediction hyperplane.

4. Working Process — Step by Step

Gradient Descent for multiple variables follows these seven steps each iteration:

1. Input Features — Feed training data with all features and true outputs y into the model.
2. Prediction — Calculate the predicted output using the current hypothesis $h_{\theta}(x)$.
3. Error Calculation — Compute the residual for each example: $(h_{\theta}(x) - y)$.
4. Cost Function — Aggregate all errors into a single scalar $J(\theta)$ via Mean Squared Error.
5. Gradient Calculation — Compute the partial derivative $\partial J / \partial \theta_j$ for each parameter.
6. Parameter Update — Simultaneously update every θ_j using the update rule.
7. Iterate — Repeat steps 2 – 6 until $J(\theta)$ converges (stops decreasing meaningfully).

4.1 Cost Function

The cost function measures total prediction error over all m training examples:

$$J(\theta_0, \theta_1, \theta_2) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

Symbol	Meaning
m	Number of training examples
$y^{(i)}$	Actual (true) output value for the i -th example
$h_{\theta}(x^{(i)})$	Predicted value for the i -th example
$J(\theta)$	Scalar cost value — this is what we minimise

4.2 Gradient Descent Update Rule

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Substituting the partial derivative explicitly:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)}$$

Repeat until convergence:

$$\text{Repeat until convergence: } \theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}, \quad \forall j$$

4.3 Numerical Worked Example

Using: Hours Studied (x_1), Sleep Hours (x_2) $\rightarrow$ Exam Score (y). Initial values: $\theta_0 = \theta_1 = \theta_2 = 0$, learning rate $\alpha = 0.01$.

Step-by-Step Example

Suppose we want to predict exam score using:

Hours Studied (x_1)	Sleep Hours (x_2)	Score (y)
1	6	50
2	5	55
3	8	70

So: $m = 3$

Hypothesis Function $\gg h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$

Assume initial values:

- $\theta_0 = 0$ and Learning rate:
- $\theta_1 = 0$ • $\alpha = 0.01$
- $\theta_2 = 0$

Figure 2 — Dataset and initial hypothesis setup

Example	y	Prediction $h_{\theta}(x)$	Error $h_{\theta}(x) - y$
1	50	0	$0 - 50 = -50$
2	55	0	$0 - 55 = -55$
3	70	0	$0 - 70 = -70$

Step 1: Sum All Errors

$$\begin{aligned} \sum (h_{\theta}(x) - y) \\ &= -50 + (-55) + (-70) \\ &= -175 \end{aligned}$$

Step 2: Divide by m

$$\begin{aligned} \frac{1}{m} \sum (h_{\theta}(x) - y) \\ &= \frac{-175}{3} \\ &= -58.333 \end{aligned}$$

Step 3: Multiply by Learning Rate

$$\begin{aligned} \alpha \times \left(\frac{-175}{3} \right) \\ &= 0.01 \times (-58.333) \\ &= -0.58333 \end{aligned}$$

Step 4: Update θ_0

Formula: $\theta_0 = 0 - (-0.58333)$

Subtracting a negative means addition: $\theta_0 = 0 + 0.58333$

Final answer: $\theta_0 \approx 0.583$

Figure 3 — Step-by-step calculation of the θ_0 update

Training Data

Example	x_2 (Sleep Hours)	Actual y	Prediction	Error $h_{\theta}(x) - y$
1	6	50	0	-50
2	5	55	0	-55
3	8	70	0	-70

Multiply Error by x_2 $(h_{\theta}(x) - y)x_2$

Example 1 $\Rightarrow (-50)(6) = -300$

Example 2 $\Rightarrow (-55)(5) = -275$

Example 3 $\Rightarrow (-70)(8) = -560$

Figure 4 — Multiplying error by x_2 for the ϑ_2 update

Updated Parameters

After 1 iteration:

Parameter	Value
θ_0	0.583
θ_1	1.233
θ_2	3.783

New hypothesis:

$$h_{\theta}(x) = 0.583 + 1.233x_1 + 3.783x_2$$

Now predictions improve.

Gradient descent repeats this process many times until the cost becomes very small.

Figure 5 — Updated parameters after one complete iteration

After one iteration: $\vartheta_0 \approx 0.583$, $\vartheta_1 \approx 1.233$, $\vartheta_2 \approx 3.783$. The model continues iterating until the cost becomes negligibly small.

5. Gradient Descent Variants

Gradient Descent comes in three variants that differ only in how many training examples are used per parameter update. Choosing the right variant depends on dataset size, available memory, and desired convergence properties.

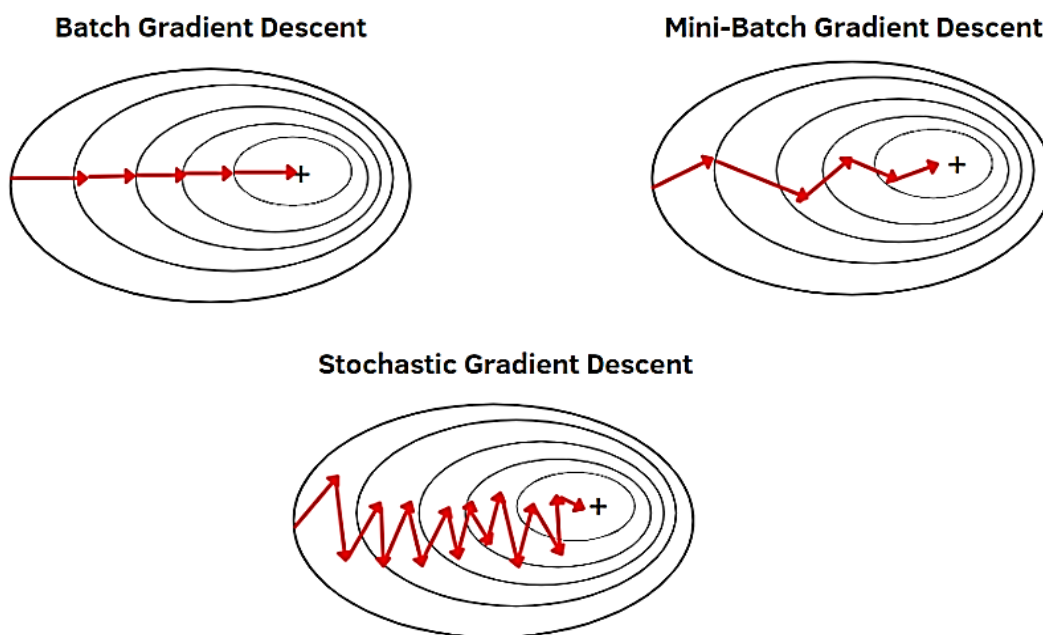


Figure 6 — Convergence paths: Batch (smooth) vs SGD (noisy) vs Mini-Batch (balanced)

5.1 Batch Gradient Descent

Definition

Batch GD computes the gradient of the cost function using the entire training dataset before making a single parameter update. One full pass over all data = one update.

Update Rule

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)}$$

Working Process

8. Load the entire training dataset into memory.
9. Calculate the prediction and residual for every training example.
10. Sum and average the gradients over all m examples.
11. Update every parameter θ_j once using the averaged gradient.

12. Repeat until $J(\theta)$ converges.

Advantages

- Smooth, stable convergence path — predictable behaviour.
- Accurate parameter updates because every example contributes.
- Guaranteed convergence to the global minimum for convex cost functions.

Disadvantages

- Requires the entire dataset in RAM — infeasible for large datasets.
- Very slow — one parameter update per full data pass.
- Cannot support online learning (new data requires restarting).
- Can get stuck in saddle points in non-convex landscapes.

Real-World Example — House Price Prediction

With 10 000 house records, Batch GD reads all 10 000 before updating weights even once. Each iteration is slow, but every update is maximally informed.

Summary Table

Property	Batch GD
Data per update	All m training examples
Memory requirement	High — full dataset in RAM
Speed per epoch	Very slow
Convergence path	Smooth and direct
Online learning	Not supported
Best for	Small datasets

5.2 Stochastic Gradient Descent (SGD)

Definition

SGD updates the model parameters after processing just one randomly selected training example at a time. "Stochastic" refers to the random selection of that one example.

Update Rule

$$\theta_j := \theta_j - \alpha \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)}$$

Notice: there is no summation, no average — one example drives the update.

Working Process

13. Shuffle the training dataset.
14. Pick one training example $(x^{(i)}, y^{(i)})$.
15. Compute prediction, residual, and gradient for that single example.
16. Immediately update all parameters θ_j .
17. Move to the next example and repeat.

Why SGD Is Faster

Batch GD makes 1 update per epoch; SGD makes m updates per epoch. Even though each update is less accurate, the model learns far more quickly overall.

Online Learning

SGD processes one example at a time, so new data can arrive continuously. This makes it ideal for real-time systems such as recommendation engines or stock-price predictors.

Escaping Local Minima

The noise in SGD updates can accidentally jump over shallow local minima, allowing the optimiser to find a better overall solution than Batch GD might.

Advantages

- Frequent updates — learns much faster than Batch GD.
- Minimal memory — only one example per step.
- Supports online learning and streaming data.
- Noisy updates can escape local minima.

Disadvantages

- High variance in updates — noisy, oscillating convergence path.
- Difficult to reach the exact minimum — oscillates around it.
- Requires careful tuning of learning rate α .

Real-World Example — Image Classification

Training a neural network on millions of images: loading all images at once is impossible. SGD processes one image at a time, continuously updating weights.

5.3 Mini-Batch Gradient Descent

Definition

Mini-Batch GD divides the dataset into small batches (typically 32 – 256 examples) and updates parameters after each batch. It is the most widely used variant in practice, including all major deep-learning frameworks.

Update Rule

$$\theta_j := \theta_j - \alpha \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)}$$

where $\mathcal{B}$ is a mini-batch and $|\mathcal{B}|$ is the batch size.

Working Process

18. Divide the dataset into mini-batches of size $|\mathcal{B}|$ (e.g., 32).
19. For each mini-batch: compute predictions, residuals, and averaged gradient.
20. Update all parameters θ_j .
21. Move to the next mini-batch until the full dataset is processed (= 1 epoch).
22. Repeat for multiple epochs until convergence.

Why It Combines the Best of Both

- From Batch GD: more accurate, more stable updates than pure SGD.
- From SGD: memory-efficient, much faster than full Batch GD.
- From hardware: exploits GPU parallelism — multiple examples processed simultaneously.

Advantages

- Efficient GPU/CPU utilisation via batch matrix operations.
- Stable convergence — less noisy than SGD.
- No need to hold the entire dataset in RAM.
- Best generalisation performance in practice.

Disadvantages

- Requires tuning of the batch-size hyperparameter.
- Slightly less accurate per update than full Batch GD.

TensorFlow and PyTorch default to Mini-Batch GD. Typical batch sizes: 32, 64, 128, 256 — powers of 2 for GPU efficiency.

6. Comparison of Gradient Descent Algorithms

Property	Batch GD	SGD	Mini-Batch GD
Data per update	All m examples	1 example	$ \mathcal{B} $ examples (32–256)
Speed	Slow	Fast (m updates/epoch)	Fastest in practice
Memory usage	High (full dataset)	Very low (1 sample)	Low to medium
Accuracy per update	High	Low (noisy)	Medium–high
Convergence path	Smooth, direct	Noisy, zigzag	Slightly noisy, stable
Stability	Very stable	Unstable / noisy	Stable
Escapes local minima	No	Yes (noise)	Partial
Online learning	No	Yes	Yes
GPU efficiency	Low	Low	High
Best dataset size	Small only	Large / streaming	All sizes
Common usage	Textbook examples	Real-time systems	Deep learning standard

7. Gradient Descent Update Rule

7.1 The Formula

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

7.2 Each Component Explained

Symbol	Name	Role in the Update
θ_j	Parameter	The weight currently being updated
α	Learning rate	Controls the step size; chosen by the practitioner
$\partial J / \partial \theta_j$	Gradient	Direction of steepest increase of cost — we subtract it to descend
$J(\theta)$	Cost function	Total prediction error to be minimised

7.3 The Partial Derivative (Gradient)

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$$

7.4 Intuition — Rolling Downhill

Imagine the cost function $J(\theta)$ as a hilly landscape. Gradient Descent is like placing a ball on a hill and letting it roll to the lowest valley:

- The gradient tells which direction is "uphill" (increasing cost).
- Subtracting the gradient moves "downhill" (decreasing cost).
- The learning rate α controls how large each step is.

7.5 Learning Rate Behaviour

α too large $\Rightarrow$ overshoots minimum; α too small $\Rightarrow$ slow convergence

Learning Rate α	Effect	Outcome
Too large	Steps overshoot the minimum	Cost diverges (increases)
Too small	Steps are tiny	Very slow convergence
Just right	Steps are well-sized	Converges efficiently

Critical rule: ALL parameters $\vartheta_0, \vartheta_1, \vartheta_2, \dots$ must be updated simultaneously using the OLD values. Never use a freshly updated parameter in the same iteration.

8. Bias

8.1 Definition

Bias is the error that arises from wrong assumptions in the learning algorithm. A biased model oversimplifies reality and fails to capture the true underlying pattern in the data. It is defined as the difference between the expected prediction and the true value.

- ✓ **Bias** refers to the errors which occur when we try to fit a statistical model on real-world data which does not fit perfectly well on some mathematical model. It may be defined by the differences or error occurring between the model's **predicted value and the actual value**.
- ✓ Let Y be the true value of a parameter; and let $\hat{Y}$ be an estimator of Y based on a sample of data. Then, the bias of the estimator $\hat{Y}$ is given by:
 - $\text{Bias}(\hat{Y}) = E(\hat{Y}) - Y$
- ✓ where $E(\hat{Y})$ is the expected value of the estimator $\hat{Y}$. It is the measurement of the model that how well it fits the data.
- ✓ The high-bias model will not be able to capture the dataset trend. It is considered as the underfitting model which has a high error rate. It is due to a very simplified algorithm.
- ✓ For example, a linear regression model may have a high bias if the data has a non-linear relationship.
- ✓ We can use a more complex model like Polynomial regression for non-linear datasets

Figure 7 — Bias definition from lecture slides

8.2 Mathematical Definition

$$\text{Bias}(\hat{Y}) = E[\hat{Y}] - Y$$

Symbol	Meaning
$\hat{Y}$ (Y-hat)	The estimator / model's predicted value
$E[\hat{Y}]$	Expected value of $\hat{Y}$ — average prediction over many datasets
Y	The true actual value
$\text{Bias}(\hat{Y})$	Systematic gap between expected prediction and truth

8.3 Formal Bias² Definition

$$\text{Bias}^2[\hat{f}(x)] = \left(E[\hat{f}(x)] - f(x) \right)^2$$

Here $\hat{f}(x)$ is the model's prediction and $f(x)$ is the true function.

8.4 High Bias

A model with high bias makes overly simple assumptions. It systematically underpredicts or overpredicts, regardless of the training data seen — this is called underfitting.

Example: Linear Regression on Non-Linear Data

If the true relationship is curved (polynomial) but we force a straight line, the model has high bias. No matter how much data we use, the line cannot capture the curve.

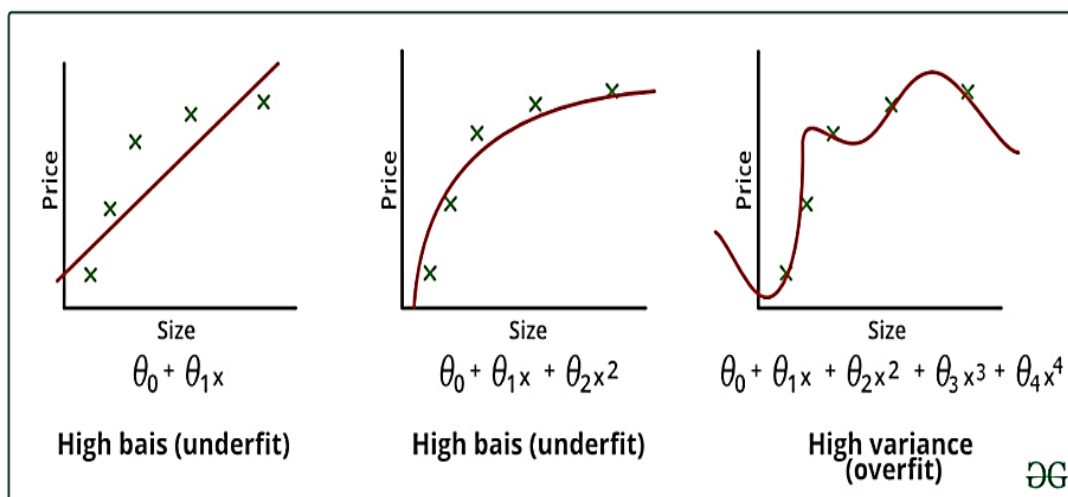


Figure 8 — High bias (underfit) vs high variance (overfit) illustrated on house price data

8.5 Characteristics of High-Bias Models

Characteristic	Description
Training error	High — the model misses even training patterns
Test error	High — poor generalisation to new data
Model complexity	Too low — e.g., linear model on curved data
Error type	Underfitting
Fix	Use a more complex model; add more features

9. Variance

9.1 Definition

Variance measures how much a model's predictions change when it is trained on different subsets of the training data. A high-variance model is overly sensitive to small fluctuations or noise in the training set.

9.2 Formal Definition

$$\text{Variance}[\hat{f}(x)] = \mathbb{E}\left[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2\right]$$

9.3 Sensitivity to Training Data

If you train the same model architecture on two slightly different training sets and obtain very different prediction curves, the model has high variance — it is memorising training data rather than learning the underlying pattern.

9.4 Why Variance Occurs

- Model is too complex (high-degree polynomial with many parameters).
- Model is trained on too little data.
- No regularisation is applied.
- Model captures noise in the training data as if it were a real signal.

9.5 High Variance = Overfitting

A high-variance model fits training data perfectly but fails on any unseen example. Training error is very low; test error is very high.

9.6 Characteristics of High-Variance Models

Characteristic	Description
Training error	Very low — fits every point
Test error	Very high — cannot generalise
Model complexity	Too high — wiggly, over-parameterised
Error type	Overfitting
Fix	Regularisation; more training data; simpler model

10. Bias vs Variance

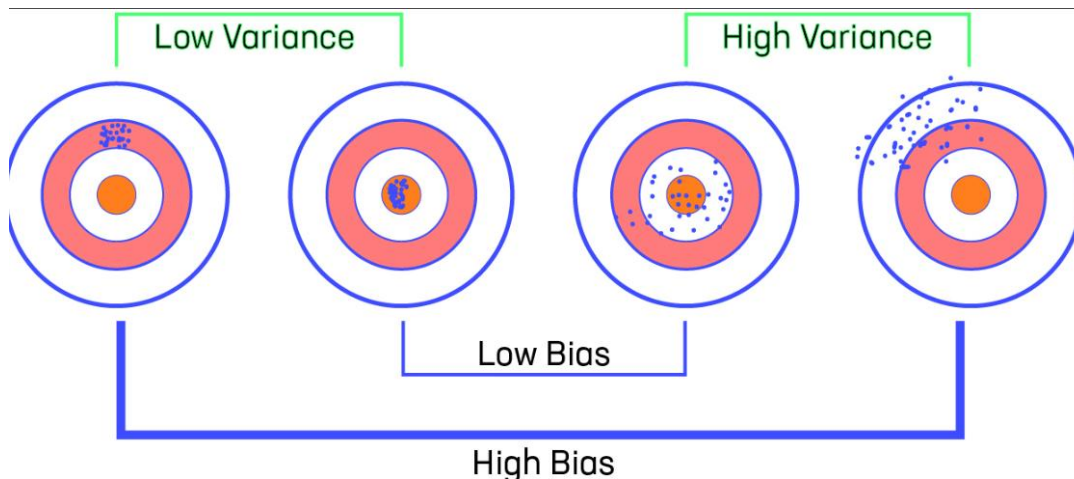


Figure 9 — Target diagram: Low/High Bias × Low/High Variance

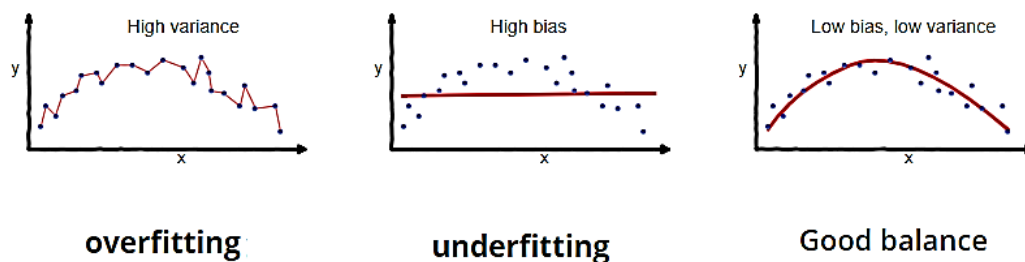


Figure 10 — Overfitting, Underfitting, and Good Balance

10.1 Key Definitions

Term	Definition
Low Bias	Average prediction is very close to the true value
High Bias	Predictions are consistently and systematically wrong
Low Variance	Predictions are tightly clustered — consistent across datasets
High Variance	Predictions are widely scattered — change a lot with different training sets

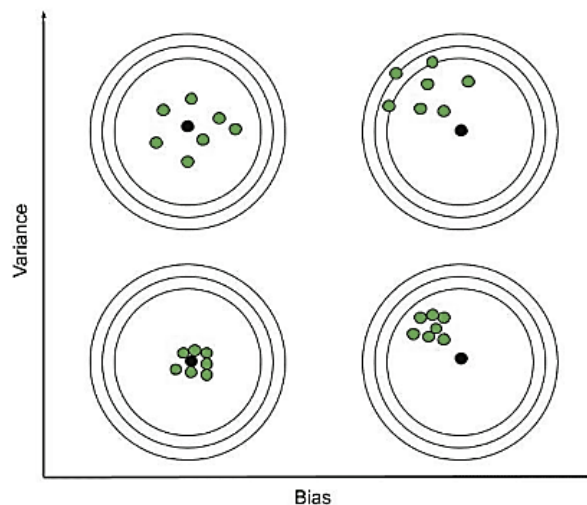
10.2 Why Both Matter

Both bias and variance are sources of prediction error. The ideal model has low error on unseen data, which requires both: accurate average predictions (low bias) and stable predictions regardless of the specific training set (low variance).

10.3 Model Complexity and the Tradeoff

As model complexity increases (more parameters, higher-degree polynomial): bias decreases but variance increases. As complexity decreases: bias increases but variance decreases. There is a sweet spot — the optimal model complexity.

11. Four Bias-Variance Cases



Bias-Variance Combinations

Figure 11 — All four bias-variance combinations on a target diagram

Case 1 — High Bias + Low Variance (Underfitting)

The model consistently predicts in the wrong direction but the predictions are tightly clustered. It is too simple to capture the real pattern.

- Training error: High
- Test error: High
- Problem: Underfitting — model is too simple.
- Fix: Use a more complex model or add more features.

Case 2 — Low Bias + High Variance (Overfitting)

The model memorises the training data and predicts it almost perfectly, but predictions on new data are scattered and unreliable.

- Training error: Very low
- Test error: Very high
- Problem: Overfitting — model is too complex.
- Fix: Regularisation; more training data; reduce model complexity.

Case 3 — High Bias + High Variance (Worst Case)

Predictions are both systematically wrong (high bias) and inconsistent (high variance). This is the worst possible model — it fails on both training and test data, and in different ways each time.

- Training error: High

- Test error: Very high
- Fix: Fundamental redesign of model and/or data preprocessing.

Case 4 — Low Bias + Low Variance (Ideal)

The ideal goal. Predictions are accurate on average (low bias) and consistent across different training datasets (low variance). The model generalises well to unseen data.

- Training error: Low
- Test error: Low and close to training error

In practice, perfectly achieving Low Bias + Low Variance is impossible. We use regularisation, cross-validation, and ensemble methods to get as close as possible.

12. Bias-Variance Tradeoff

12.1 Definition

The Bias-Variance Tradeoff is the fundamental tension between a model's ability to fit training data (low bias) and its ability to generalise to new data (low variance). Reducing one tends to increase the other.

12.2 Total Error Decomposition

$$\text{Total Error} = \underbrace{\text{Bias}^2}_{\text{underfit}} + \underbrace{\text{Variance}}_{\text{overfit}} + \underbrace{\epsilon}_{\text{irreducible noise}}$$

Irreducible noise (ϵ) is inherent randomness in any real-world dataset — it cannot be eliminated regardless of the model used. We can only minimise $\text{Bias}^2 + \text{Variance}$.

12.3 Why the Tradeoff Exists

A simple model (straight line) cannot capture complex patterns → high bias. A complex model (high-degree polynomial) fits every training point perfectly → high variance. There is no single model that is both perfectly accurate on average and perfectly stable.

12.4 Model Tuning Strategy

Symptom Observed	Root Cause	Recommended Fix
High training error + High test error	High Bias (Underfit)	More complex model; add features; reduce regularisation
Low training error + High test error	High Variance (Overfit)	Regularisation; more data; simpler model; dropout
High training error + Very high test error	High Bias + High Variance	Redesign model architecture; clean and augment data

12.5 Practical Importance

- Guides every modelling decision: model selection, feature engineering, regularisation.
- Cross-validation estimates whether the model suffers more from bias or variance.
- Regularisation is the most practical tool for managing the tradeoff.

Key insight: Regularisation deliberately introduces a small amount of bias in exchange for a much larger reduction in variance. The net effect is a better-performing model on unseen data.

13. Correlation

13.1 Definition

Correlation measures the statistical relationship between two variables. In machine learning, correlation analysis guides feature selection and reveals how strongly each feature relates to the target variable.

13.2 Types of Correlation

Type	Description	Plot Pattern	Example
Positive	Both variables increase together	Upward trend	Hours studied → Exam score
Negative	One increases, the other decreases	Downward trend	Sleep deprivation → Performance
Zero / None	No predictable relationship	Random scatter	Shoe size → Intelligence

13.3 Interpretation

Correlation does not imply causation. Two features may be highly correlated without either causing the other (e.g., ice-cream sales and drowning incidents both rise in summer — the real cause is temperature). In ML, correlated features can cause multicollinearity, making coefficient estimates unstable. Ridge Regression handles multicollinearity well.

14. Regularization

14.1 Definition

Regularisation is a technique that prevents overfitting by adding a penalty term to the cost function. This penalty discourages the model from assigning very large values to its parameters, thereby reducing model complexity.

14.2 Why Regularisation Is Needed

Overfitting occurs when a model is too complex — it memorises the training set and fails on new data. Regularisation controls this by penalising large parameter values, forcing the model to stay simple even if a complex equation form is used.

14.3 How It Works

Penalising Large Coefficients

Without regularisation the optimiser freely sets any parameter value. With regularisation large parameter values increase the total cost, so the optimiser is forced to keep them small.

Reducing Model Complexity

Smaller parameter values mean the model relies less on individual features. This reduces the tendency to fit noise rather than signal.

Improving Generalisation

A regularised model trades a small amount of training accuracy for much better test accuracy. This trade-off is almost always worthwhile.

14.4 Benefits of Regularisation

- Prevents overfitting.
- Improves performance on test / unseen data.
- Can implicitly perform feature selection (Lasso).
- Works well for high-dimensional data (many features).
- Produces more stable, reliable models.
- Helps achieve a better balance between bias and variance.

15. Min-Max Normalization

15.1 Definition

Min-Max Normalisation (also called feature scaling) transforms all values of a feature to a fixed range, typically $[0, 1]$. It ensures every feature contributes equally to the gradient regardless of its original scale.

15.2 Formula

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1]$$

Symbol	Meaning
x	Original feature value
$x_{\min}$	Minimum value in that feature column
$x_{\max}$	Maximum value in that feature column
x_{norm}	Scaled value in $[0, 1]$

15.3 Why Feature Scaling Is Needed

In Gradient Descent, if features have very different scales (e.g., house size in thousands vs number of rooms as single digits), the cost surface becomes elongated and narrow. The gradient takes tiny steps in one direction and huge steps in another, causing slow, oscillating convergence. Normalisation makes the cost surface more circular and convergence much faster.

15.4 Benefits

- Dramatically faster Gradient Descent convergence.
- Prevents large-scale features from dominating smaller ones.
- Required for algorithms sensitive to scale: KNN, SVM, Neural Networks.
- Improves numerical stability of the optimiser.

15.5 Worked Example

Original house sizes: $[500, 1000, 1500, 2000]$ sq ft

$x_{\min} = 500$, $x_{\max} = 2000$

Normalised values:

- $500 \rightarrow (500 - 500) / (2000 - 500) = 0.00$

- $1000 \rightarrow (1000 - 500) / (2000 - 500) = 0.33$
- $1500 \rightarrow (1500 - 500) / (2000 - 500) = 0.67$
- $2000 \rightarrow (2000 - 500) / (2000 - 500) = 1.00$

16. Overfitting

16.1 Definition

Overfitting occurs when a model learns the training data too well — including noise and random fluctuations — and fails to perform on new, unseen data. The model memorises rather than generalises.

16.2 House Price Example

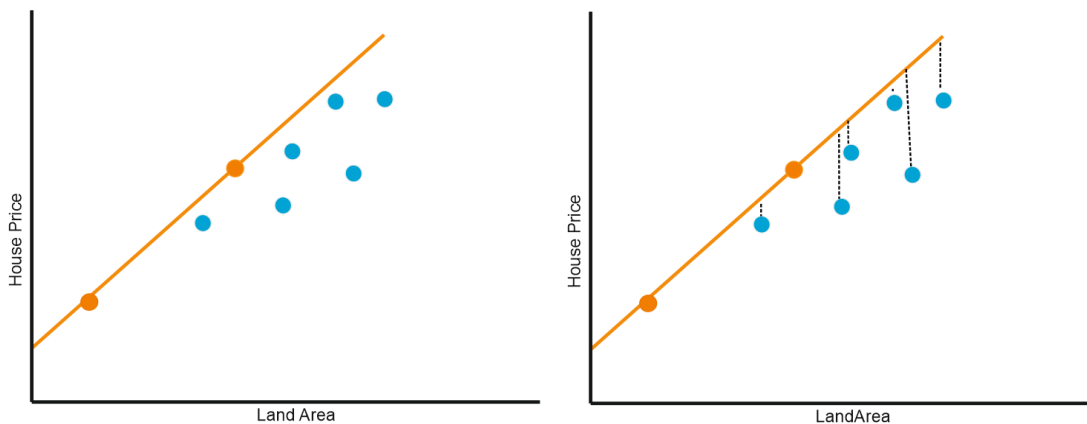


Figure 12 — Overfitting: trained on 2 orange points, fails on new blue points

Using only 2 training points (orange), a complex polynomial fits them perfectly. When tested on new data (blue dots), predictions are wildly wrong. The model has very low training error but very high test error.

16.3 Why Overfitting Happens

- Model is too complex for the available data.
- Training dataset is too small.
- Model trained for too many iterations without early stopping.
- No regularisation is applied.

16.4 Signs of Overfitting

Sign	Description
Training accuracy very high	Model fits every training point
Test accuracy much lower	Model fails on unseen data
Large train-test gap	Performance drops sharply outside training
Wiggly, complex decision boundary	Model follows noise, not trend

Sign	Description
High sensitivity to input	Small change in input → large change in prediction

16.5 Problems Caused by Overfitting

- Model is useless in production — fails on any real-world data.
- Creates false confidence — high training accuracy is misleading.
- Wastes computational resources on an unreliable model.

17. Underfitting

17.1 Definition

Underfitting occurs when a model is too simple to capture the underlying patterns in the data. The model fails to learn from training data and performs poorly on both training and test data.

17.2 Causes

- Model is too simple (e.g., linear model for non-linear data).
- Too few features provided.
- Insufficient training iterations.
- Over-regularisation — penalty is so strong it prevents learning.
- Poor feature engineering.

17.3 Examples

- Fitting a straight line to data that follows a parabola.
- Using only 1 feature to predict house price when 10 features are available.
- Using a decision stump (depth-1 tree) on a complex classification problem.

17.4 Overfitting vs Underfitting — Full Comparison

Property	Overfitting	Underfitting
Training error	Very low	High
Test error	Very high	High
Train-test gap	Large	Small (both high)
Model complexity	Too high	Too low
Bias	Low	High
Variance	High	Low
Problem type	Memorises training data	Cannot learn patterns
Solution	Regularisation, more data	More complex model, more features
Other name	High-variance problem	High-bias problem

18. Why Regularization?

18.1 Overfitting Solution

When a model overfits, its parameters take on very large values that create a complex, wiggly prediction surface. Regularisation directly penalises these large values, forcing the model to become simpler and more general.

18.2 Complexity Reduction

By adding a penalty proportional to the size of parameters, regularisation limits the effective complexity of the model, even if a high-degree equation form is used. The model becomes less sensitive to individual data points.

18.3 Better Generalisation

A regularised model trades a small amount of training accuracy for substantially better test accuracy. Since the goal of ML is to perform well on real-world data — not just the training set — this trade-off is almost always worthwhile.

18.4 Balance Between Bias and Variance

Regularisation increases bias slightly (the model is less flexible) but significantly reduces variance (predictions are more consistent). The net effect is a more reliable model.

Key principle: A slightly biased but consistent model is almost always more useful than one that is sometimes exactly right but unpredictable on new data.

19. Regularization Techniques

19.1 Ridge Regression (L2 Regularization)

Definition

Ridge Regression adds the sum of the squared parameter values as a penalty to the cost function. This is the L2 norm penalty.

Cost Function with Ridge Penalty

$$J(\theta) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2}_{\text{MSE}} + \lambda \sum_{j=1}^n \theta_j^2$$

Symbol	Meaning
MSE	Mean Squared Error — original prediction cost
λ (lambda)	Regularisation strength hyperparameter — controls penalty size
$\sum \theta_j^2$	L2 penalty — sum of squared parameter values

How It Works

Ridge adds the squared magnitude of all coefficients to the loss. This forces the optimiser to keep all coefficients small. Crucially, Ridge never sets a coefficient exactly to zero — all features remain active in the model but with reduced influence.

Advantages

- Effectively prevents overfitting.
- Works well when all features genuinely contribute to the prediction.
- Handles multicollinearity — distributes weight evenly among correlated features.
- Mathematically smooth — has a unique, closed-form solution.

Disadvantages

- Cannot perform feature selection — all features remain in the model.
- Less interpretable with many features.
- Choosing the right λ requires cross-validation.

Use Cases

- When all features are relevant and should be kept.

- When features are highly correlated (multicollinearity).
- Regression problems with many features.

19.2 Lasso Regression (L1 Regularization)

Definition

Lasso Regression adds the sum of the absolute values of all parameters as a penalty. LASSO stands for Least Absolute Shrinkage and Selection Operator.

Cost Function with Lasso Penalty

$$J(\theta) = \underbrace{\frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2}_{\text{MSE}} + \lambda \sum_{j=1}^n |\theta_j|$$

Symbol	Meaning
MSE	Mean Squared Error — original prediction cost
λ (lambda)	Regularisation strength hyperparameter
$\sum \theta_j $	L1 penalty — sum of absolute parameter values

How It Works

Lasso adds the absolute magnitude of coefficients to the loss. Unlike Ridge, Lasso can shrink some coefficients all the way to exactly zero, completely removing those features from the model.

Feature Selection Property

Setting coefficients to exactly zero is Lasso's most important advantage. This performs automatic feature selection: the model retains only the most predictive features and discards the rest.

Advantages

- Automatic feature selection — eliminates irrelevant features.
- Produces sparse, more interpretable models.
- Ideal when only a few features are truly predictive.
- Better for high-dimensional datasets with many irrelevant features.

Disadvantages

- With correlated features, Lasso arbitrarily picks one and discards the rest.
- Less effective when all features genuinely matter.
- Not mathematically smooth at zero — harder to optimise than Ridge.

Use Cases

- Feature selection in high-dimensional datasets.
- When you expect many irrelevant or redundant features.
- Building interpretable, sparse models.

20. Ridge vs Lasso Regression

Property	Ridge Regression (L2)	Lasso Regression (L1)
Penalty formula	$\lambda \sum \theta_j^2$ (sum of squares)	$\lambda \sum \theta_j $ (sum of absolutes)
Coefficient behaviour	Shrinks toward zero; never exactly zero	Can shrink to exactly zero
Feature selection	No — all features kept	Yes — irrelevant features removed
Model sparsity	Dense (all features active)	Sparse (few features active)
Interpretability	Harder with many features	Easier — fewer active terms
Multicollinearity	Handled well — distributes weight	Less stable — selects one feature
Mathematical smoothness	Smooth — unique closed-form solution	Not smooth at zero
Bayesian equivalent	Gaussian prior on parameters	Laplace prior on parameters
Best use case	All features are relevant	Only a few features are important
λ effect	Higher $\lambda \rightarrow$ more shrinkage	Higher $\lambda \rightarrow$ more features zeroed out

Rule of thumb: Use Ridge when you believe all features contribute. Use Lasso when you expect many irrelevant features and want automatic selection.

21. Quick Revision Notes

21.1 Key Definitions

Term	One-Line Definition
Gradient Descent	Iterative algorithm that minimises cost by moving parameters in the direction of the negative gradient
Cost Function $J(\theta)$	Scalar that measures total prediction error across all training examples
Learning Rate α	Step size for each parameter update; must be tuned carefully
Hypothesis $h\theta(x)$	The model's prediction function
Bias	Systematic error from wrong model assumptions; causes underfitting
Variance	Sensitivity to training data; causes overfitting
Overfitting	Model memorises training data; fails on new data (high variance)
Underfitting	Model too simple; fails on both training and test data (high bias)
Regularisation	Technique that penalises large parameters to prevent overfitting
Ridge (L2)	Regularisation via squared penalty; shrinks all coefficients
Lasso (L1)	Regularisation via absolute penalty; can zero out coefficients
Min-Max Normalisation	Scales features to $[0, 1]$ for equal gradient contribution
Batch GD	Updates parameters using all m training examples per step
SGD	Updates parameters using one training example per step
Mini-Batch GD	Updates parameters using a small batch per step (standard in DL)

21.2 Important Formulas

Hypothesis (n features):

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Cost Function:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

Gradient Descent Update Rule:

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

Expanded Update Rule:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right) \cdot x_j^{(i)}$$

Bias Formula:

$$\text{Bias}(\hat{Y}) = \mathbb{E}[\hat{Y}] - Y$$

Variance Formula:

$$\text{Variance}[\hat{f}(x)] = \mathbb{E} \left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)] \right)^2 \right]$$

Total Error Decomposition:

$$\text{Total Error} = \underbrace{\text{Bias}^2}_{\text{underfit}} + \underbrace{\text{Variance}}_{\text{overfit}} + \underbrace{\varepsilon}_{\text{irreducible noise}}$$

Ridge Cost:

$$J(\theta) = \underbrace{\frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2}_{\text{MSE}} + \lambda \sum_{j=1}^n \theta_j^2$$

Lasso Cost:

$$J(\theta) = \underbrace{\frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2}_{\text{MSE}} + \lambda \sum_{j=1}^n |\theta_j|$$

Min-Max Normalisation:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1]$$

21.3 Key Concepts Summary

- Gradient Descent finds the cost minimum by iteratively moving in the direction of the negative gradient.
- Three GD variants differ only in how many examples drive each update: All (Batch) · One (SGD) · Mini-Batch.
- Bias = systematic error. Variance = sensitivity to training data.
- Underfitting = high bias + low variance. Overfitting = low bias + high variance.
- Regularisation reduces complexity by penalising large parameters — λ controls strength.
- Ridge keeps all features. Lasso performs automatic feature selection by zeroing irrelevant ones.
- Feature scaling (normalisation) is essential for Gradient Descent to converge efficiently.
- The ideal model has Low Bias + Low Variance — in practice we balance, not eliminate.

21.4 Comparison Quick Reference

Comparison	Option A	Option B
GD updates per epoch	Batch GD: 1 update	SGD: m updates
GD memory	Batch: entire dataset in RAM	SGD: 1 example only
GD convergence	Batch: smooth path	SGD: noisy/zigzag path
Fitting problem	Overfitting: low train, high test error	Underfitting: high both
Error source	High Bias → underfit	High Variance → overfit
Penalty type	L1 (Lasso): $ \theta_j $	L2 (Ridge): θ_j^2
Feature selection	Ridge: No (all features kept)	Lasso: Yes (zeroes out features)

21.5 Exam Questions

23. What is Gradient Descent? How does it minimise the cost function?
24. What is the difference between Batch GD, SGD, and Mini-Batch GD?
25. Define bias and variance. How do they relate to underfitting and overfitting?
26. Explain the Bias-Variance Tradeoff. Why cannot both be zero simultaneously?

27. What is regularisation? Why is it needed? How does it work?
28. What is the difference between Ridge and Lasso Regression?
29. Which regularisation technique performs feature selection, and why?
30. How do you detect overfitting? How do you fix it?
31. What happens when the learning rate α is too large? Too small?
32. Why is Min-Max Normalisation important for Gradient Descent?

21.6 Viva Q&A

Viva Question	Short Answer
What does the cost function measure?	Total prediction error across all m training examples.
What happens if α is too large?	GD overshoots the minimum — cost diverges instead of decreasing.
What is a batch in Mini-Batch GD?	A small subset (e.g., 32–256 examples) used for one parameter update.
Why does SGD have noisy convergence?	Each update uses only one example, so each gradient estimate is imprecise.
What does high bias indicate?	The model is too simple — it underfits and cannot capture the real pattern.
What does high variance indicate?	The model is too complex — it overfits and memorises training data.
How does Lasso select features?	The L1 penalty can shrink coefficients to exactly zero, removing those features.
Why is Ridge better for correlated features?	Ridge distributes weight among correlated features; Lasso arbitrarily picks one.
What is the main goal of regularisation?	Reduce model complexity and prevent overfitting by penalising large coefficients.
What is the ideal ML model goal?	Low Bias + Low Variance — accurate average predictions and consistent across datasets.
When to choose Ridge vs Lasso?	Ridge: all features matter. Lasso: only a few features are truly important.
What does λ control in regularisation?	Regularisation strength. Higher λ = stronger penalty = simpler model.
What is Min-Max normalisation for?	Scale features to $[0,1]$ so all features contribute equally to the gradient.
What is online learning in SGD?	SGD can continuously learn from newly arriving data without restarting training.

End of Study Note — Machine Learning Part 3

Good Luck with Your Exams!